

Atelier 2 : charger les relevés et faire le bilan des rejets

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python pour la science des données · **Chapitre** : 2, Fichiers et erreurs · **Version** : 2026-10-23

Avant de commencer

Ce que vous allez construire	Un notebook qui charge les 52 021 relevés du PAM **en Python pur** (sans pandas), calcule l'évolution du prix du riz importé à Dakar, puis traite un fichier d'enquête abîmé : rejets par type d'erreur, corrections sûres, bilan enregistré en JSON
Durée	🟢 Débutant : 3 h 30 · 🔵 Standard : 2 h 30 · 🟣 Expert : 1 h 45 (sans les défis)
Prérequis	Avoir lu le chapitre 2 ; votre dépôt `prix-marches` de l'atelier 1
Fichiers fournis	`wfp_food_prices_sen.csv` (les relevés du PAM), `releves_enquete.csv` (une enquête de 300 relevés saisis à la main), le notebook de départ
Rendu	Le notebook `atelier2_fichiers.ipynb` dans votre dépôt `prix-marches`, avec l'étiquette `atelier-2`

> 📍 **Point de départ** : le notebook de départ fourni, une section par étape. Envoyez les deux fichiers CSV dans Colab.

Le fichier d'enquête imite une saisie faite sur le terrain par des enquêteurs : les prix viennent des vrais relevés de 2025, mais des erreurs de saisie y ont été ajoutées. Ses colonnes : `date`, `region`, `marche`, `denree`, `unite`, `prix`.

Comment travailler

Mêmes règles que l'atelier 1 : une cellule de texte avant chaque cellule de code, une cellule de texte pour chaque réponse aux questions 💡 **A comprendre**. **Pas de pandas** dans cet atelier : uniquement `csv`, `json`, `datetime` et Python.

Étape 1 : charger les 52 021 relevés en Python pur (25 min)

🎯 **Objectif** : une liste de relevés avec les bons types.

🛠️ **A faire**

1. Chargez le fichier du PAM avec `csv.DictReader` dans une liste `brut` ; affichez son nombre de relevés.
2. Écrivez `convertir_pam(ligne)`, qui renvoie un dictionnaire avec les clés `date` (un objet `date`), `region` (la colonne `admin1`), `marche`, `denree`, `unite` et `prix` (un `float`).
3. Convertissez tous les relevés, puis gardez dans `au_kilo` seulement ceux dont l'unité est `KG`.

✅ **Vérification** : 52 021 relevés lus ; 51 854 au kilo ; 167 écartés (le bétail, par tête). Le premier : `{'date': datetime.date(2000, 1, 15), 'region': 'Dakar', 'marche': 'Tilene', 'denree': 'Maize (imported)', 'unite': 'KG', 'prix': 149.0}`.

💡 **A comprendre** : pourquoi n'utilise-t-on pas de `try` / `except` dans `convertir_pam` ? (Que sait-on de la qualité de ce fichier ?)

Étape 2 : le riz importé à Dakar, année par année (30 min)

🎯 **Objectif** : une première vraie question, en Python pur.

🛠️ **A faire**

1. Un dictionnaire `par_annee` : pour chaque année, la liste des prix du riz importé (`Rice (imported)`) dans la région de Dakar. Indice : `par_annee.setdefault(r["date"].year, []).append(r["prix"])`.
2. Affichez, de 2018 à 2026, le prix moyen et le nombre de relevés.
3. Calculez la hausse entre 2018 et 2025, en pourcentage.

✅ **Vérification**

Année	2018	2019	2020	2021	2022	2023	2024	2025	2026
Prix moyen (F/kg)	313,2	315,0	306,5	300,0	353,7	340,8	407,7	458,5	401,6

Hausse 2018-2025 : **46 %**.

💡 **A comprendre**

- En quelle année le prix commence-t-il à monter nettement ? Cherchez ce qui s'est passé dans le monde cette année-là (prix des céréales, transport maritime).
- La valeur 2026 est plus basse : peut-on comparer une année complète à une année qui s'arrête en juillet ? Combien de relevés y a-t-il en 2026 ?

Étape 3 : enregistrer le résultat en JSON (15 min)

🛠️ **A faire** : enregistrez dans `riz_dakar.json` un dictionnaire avec la source des données (« World Food Programme, via HDX (CC BY-IGO) »), la denrée, la région, et le prix moyen par année de 2018 à 2026 (arrondi à 1 décimale). Relisez le fichier et affichez la valeur de 2025.

✅ **Vérification** : `458.5`.

💡 **A comprendre** : les années sont des clés `str(annee)` dans le JSON. Que se passerait-il avec des clés `int` à la relecture ? (Essayez.)

Étape 4 : le fichier d'enquête abîmé (40 min)

🎯 **Objectif** : lire les 300 relevés sans jamais planter, avec une raison **simple** pour chaque rejet.

🛠️ **A faire**

1. Écrivez `convertir_enquete(ligne)`, qui lève une `ValueError` avec l'une de ces raisons, testées dans cet ordre : `colonne manquante`, `unité inconnue` (seul `KG` est accepté), `date illisible`, `prix manquant` (texte vide), `prix illisible`, `prix négatif ou nul`.
2. Une boucle qui range les relevés valides dans `valides`, et les rejets dans `rejets` (numéro de ligne, raison, contenu de la ligne).
3. Comptez les rejets **par raison**, du plus fréquent au moins fréquent.

▼ 💡 Indice : remplacer le message de Python par le vôtre

```
try:
    jour = date.fromisoformat(ligne["date"])
except ValueError:
    raise ValueError("date illisible")
```

On attrape l'erreur de Python, et on en lève une autre avec **notre** raison, plus simple à compter.

✅ **Vérification** : 272 valides, 28 rejetés ; prix illisible 11, date illisible 6, prix manquant 4, unité inconnue 3, colonne manquante 2, prix négatif ou nul 2.

💡 **A comprendre** : quelle erreur est la plus fréquente ? Quelle consigne donneriez-vous aux enquêteurs pour l'éviter ?

Étape 5 : corriger ce qui est sûr (30 min)

🎯 **Objectif** : récupérer les relevés dont l'erreur est certaine, et les compter.

🛠️ **A faire** : écrivez `nettoyer(ligne)`, qui renvoie une **copie** de la ligne et un booléen `corrige` : la virgule du prix est remplacée par un point, et une date `JJ/MM/AAAA` est réécrite `AAAA-MM-JJ`. Refaites la boucle de l'étape 4 en nettoyant chaque ligne complète avant de la convertir, et comptez les relevés corrigés **et acceptés**.

✅ **Vérification** : 286 valides, dont **14** corrigés ; 14 rejetés.

💡 **A comprendre**

- Quelles raisons de rejet restent ? Pourquoi ne peut-on pas les corriger ?
- Un prix `300,5` est-il forcément 300,5 F ? Imaginez une autre interprétation. Pourquoi faut-il dire dans le bilan combien de lignes ont été corrigées ?

Étape 6 : le bilan (20 min)

🛠️ **A faire**

1. Un dictionnaire `bilan` : nombre de relevés lus, valides, corrigés, rejetés, et taux de rejet (arrondi à 3 décimales) ; enregistrez-le dans `bilan_enquete.json`.
2. Les rejets dans `rejets_enquete.csv` (colonnes `ligne` et `raison`).
3. Une cellule de texte de 5 lignes : ce que vous diriez au responsable de l'enquête.

✅ **Vérification** : `{'lus': 300, 'valides': 286, 'corriges': 14, 'rejetes': 14, 'taux_de_rejet': 0.047}`.

Puis : **Exécution > Redémarrer la session et tout exécuter** ; si tout passe, téléchargez le notebook, déposez-le dans votre dépôt et étiquetez `atelier-2`.

🏆 Défis (facultatifs)

1. **Le riz local.** Refaites l'étape 2 pour le riz local (`Rice (local)`) à Dakar. Le riz local a-t-il augmenté autant que le riz importé ?
2. **La région la plus chère.** Pour le mil (`Millet`) en 2025, quelle région a le prix moyen le plus élevé ? Le plus bas ?
3. **Les lignes illisibles.** Affichez le contenu des 11 lignes « prix illisible » de l'étape 4 : combien sont des virgules, combien des `n.d.` ?
4. 🟣 **Les deux encodages.** Écrivez une fonction `lire_csv(chemin)` qui essaie UTF-8, puis cp1252 en cas de `UnicodeDecodeError`, et testez-la sur une copie du fichier d'enquête enregistrée en cp1252.

Grille d'autoévaluation

Je sais…	Oui	Pas encore
lire un gros fichier CSV en Python pur et convertir ses colonnes	☐	☐
vérifier les unités avant de calculer	☐	☐
regrouper des valeurs par clé avec un dictionnaire de listes	☐	☐
écarter les lignes invalides en donnant une raison simple	☐	☐
distinguer ce qu'on peut corriger de ce qu'on doit rejeter	☐	☐
enregistrer un résultat et un bilan en JSON	☐	☐